

```
For Regression
''' python
model = Sequential()
model.add(Dense(64, input_shape=(10,)), activation='relu'))
model.add(Dense(1)) # No activation
'''

## Step 4: Compile the Model (Loss Function Changes)
''' python
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
'''

### For Loss Function Based on Problem

| Problem | Loss Function | Output Activation |
|-----|-----|-----|
| Multi-class Classification | 'categorical_crossentropy' | 'softmax' |
| Binary Classification | 'binary_crossentropy' | 'sigmoid' |
| Regression | 'mean_squared_error' | None or 'linear' |

'''
'''

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder

# ----- Load and Explore Dataset -----

# Read Titanic dataset
df = pd.read_csv("Titanic-Dataset.csv")

# Display first and last 10 rows of the dataframe
df.head(10)
df.tail(10)

# Display info about data types and missing values
df.info()

# Display statistical summary of all columns
df.describe(include='all')

# Drop irrelevant columns
df.drop(columns=['PassengerId', 'Name', 'Ticket', 'Cabin'], inplace=True)

# ----- Data Preprocessing -----

# Fill missing Age values with mean
age_mean = df['Age'].mean()
df['Age'] = df['Age'].replace(np.nan, age_mean)

# Convert categorical columns to numeric using Label Encoding
le = LabelEncoder()
df['Sex'] = le.fit_transform(df['Sex']) # Male = 1, Female = 0 (usually)
df['Embarked'] = le.fit_transform(df['Embarked']) # Converts 'C', 'Q', 'S' to 0, 1, 2

# ----- Feature Selection -----

# Define features and Label
features = ['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Embarked']
label = 'Survived'

# ----- Data Visualization -----

# Plot age distribution
sns.histplot(df['Age'], bins=30)
plt.title("Age Distribution")
plt.show()

# Plot violin plot of Age vs Survival
sns.violinplot(x='Survived', y='Age', data=df)
plt.title("Age vs Survival")

# Count plot of passenger class
sns.countplot(x='Pclass', data=df)
plt.title("Passenger Class Count")
plt.show()

# Count plot of passenger class by survival
sns.countplot(x='Pclass', hue='Survived', data=df)
plt.title("Passenger Class vs Survival")
plt.show()

# ----- Model Training -----

# Drop any remaining missing values (if any)
data = df[features + [label]].dropna()

# Split features and target
X = data[features]
y = data[label]

# Create 80-20 and 70-30 splits
X_train_80, X_test_20, y_train_80, y_test_20 = train_test_split(X, y, test_size=0.2)
X_train_70, X_test_30, y_train_70, y_test_30 = train_test_split(X, y, test_size=0.3)

# Initialize Random Forest Classifiers for both splits
model_80 = RandomForestClassifier()
model_70 = RandomForestClassifier()

# Train both models
model_80.fit(X_train_80, y_train_80)
model_70.fit(X_train_70, y_train_70)

# ----- Prediction -----

# Predict on test sets
y_predict_30 = model_80.predict(X_test_20)
y_predict_20 = model_70.predict(X_test_30)

# ----- Evaluation -----

# Store evaluation results in a dictionary
results = {
    '80-20': {
        'classification_report': classification_report(y_test_20, y_predict_20, output_dict=True),
        'confusion_matrix': confusion_matrix(y_test_20, y_predict_20).tolist(),
        'accuracy_score': accuracy_score(y_test_20, y_predict_20)
    },
    '70-30': {
        'classification_report': classification_report(y_test_30, y_predict_30, output_dict=True),
        'confusion_matrix': confusion_matrix(y_test_30, y_predict_30).tolist(),
        'accuracy_score': accuracy_score(y_test_30, y_predict_30)
    }
}

# Print accuracy scores for both splits
print(results['80-20']['accuracy_score'])
print(results['70-30']['accuracy_score'])

class LibraryItem:
    def __init__(self, title, author, publication_year):
        self.title = title
        self.author = author
        self.publication_year = publication_year

    def display_info(self):
        print(self.title)
        print(self.author)
        print(self.publication_year)

class Book(LibraryItem):
    def __init__(self, title, author, publication_year, genre, number_of_pages):
        super().__init__(title, author, publication_year)
        self.genre = genre
        self.number_of_pages = number_of_pages

    def display_info(self):
        super().display_info()
        print(self.genre)
        print(self.number_of_pages)

class Magazine(LibraryItem):
    def __init__(self, title, author, publication_year, issue_number, month_of_issue):
        super().__init__(title, author, publication_year)
        self.issue_number = issue_number
        self.month_of_issue = month_of_issue

    def display_info(self):
        super().display_info()
        print(self.issue_number)
        print(self.month_of_issue)

B1 = Book("Feary meadows", "Faisal", 2025, "Adventure", 250)
B1.display_info()

students = []
language_count = {}
departments = []
languages = []

# Register students
n = int(input("How many students? "))

for i in range(n):
    name = input("Enter name: ")
    dept = input("Enter department: ")
    langs = input("Enter programming languages (comma-separated): ").split(',')

    langs = [l.strip() for l in langs]
    students.append({'name': name, 'department': dept, 'languages': langs})

if dept not in departments:
    departments.append(dept)

for lang in langs:
    if lang not in languages:
        languages.append(lang)
    if lang in language_count:
        language_count[lang] += 1
    else:
        language_count[lang] = 1

# Output
print("\n--- All Students ---")
for s in students:
    print(s)

print("\n--- Language Count ---")
for lang in language_count:
    print(lang, ":", language_count[lang])

import heapq
queue = []

def add_patient(queue, priority, name):
    heapq.heappush(queue, (priority, name))

def next_patient(queue):
    if queue:
        print("Next patient:", queue)
    else:
        print("No patients in queue.")

def treat_patient(queue):
    if queue:
        patient = heapq.heappop(queue)
        print("Treating", patient)
    else:
        print("No patient to treat")

add_patient(queue, 3, "Faisal")
add_patient(queue, 1, "Muzamil")
add_patient(queue, 5, "Saad")

next_patient(queue)
treat_patient(queue)
next_patient(queue)

# For Text (LSTM)
''' python
from keras.layers import Embedding, LSTM

model = Sequential()
model.add(Embedding(input_dim=10000, output_dim=128, input_length=100))
model.add(LSTM(64))
model.add(Dense(1, activation='sigmoid')) # For binary classification
'''

# Import pandas as pd
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
from matplotlib import pyplot as plt
from sklearn.preprocessing import MinMaxScaler, LabelEncoder, StandardScaler

# Enable inline plotting in notebooks
%matplotlib inline

# ----- Load and Explore Dataset -----

# Read the dataset
df = pd.read_csv("penguins.csv")

# Display first and last 10 rows
df.head(10)
df.tail(10)

# Display dataset info (data types, nulls, etc.)
df.info()

# Display summary statistics for all columns
df.describe(include='all')

# ----- Boxplot for Outlier Analysis -----

# Plot boxplots for numeric columns
plt.figure(figsize=(12, 8))
df[['culmen_length_mm', 'culmen_depth_mm', 'flipper_length_mm', 'body_mass_g']].boxplot()
plt.title("Boxplot of Penguin Physical Measurements")
plt.ylabel("Value")
plt.grid(True)
plt.show()

# ----- Data Preprocessing -----

# Convert categorical column 'sex' to numeric
le = LabelEncoder()
df['sex'] = le.fit_transform(df['sex'])

# Fill missing values with column-wise mean
df.fillna(df.mean(numeric_only=True), inplace=True)

# ----- Feature Scaling -----

# Standardize the dataset for clustering
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df)

# Create a new dataframe with scaled data
scaled_df = pd.DataFrame(scaled_data, columns=df.columns)

# ----- Elbow Method to Find Optimal K -----

# Try k from 1 to 10 and store sum of squared errors (inertia)
k_rng = range(1, 11)
sse = []

for k in k_rng:
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(scaled_df)
    sse.append(km.inertia_)

# Plot SSE vs K to find the "elbow"
plt.xlabel('K')
plt.ylabel('SSE')
plt.plot(k_rng, sse, marker='o')
plt.grid(True)
plt.tight_layout()
plt.show()

# ----- Final Clustering -----

# Apply KMeans with k=3 (from elbow)
k_final = KMeans(n_clusters=3, random_state=42)
predicted = k_final.fit_predict(scaled_df)

# Store predicted clusters in the original dataframe
df['cluster'] = y_predicted

# ----- Cluster Visualization -----

# Scatter plot of first two scaled features with cluster coloring
plt.figure(figsize=(8, 6))
plt.scatter(scaled_df.iloc[:, 0], scaled_df.iloc[:, 1], c=y_predicted, cmap='viridis', s=50)
plt.title("KMeans Clustering (k=3)")
plt.xlabel("Feature 1: culmen_length_mm (scaled)")
plt.ylabel("Feature 2: culmen_depth_mm (scaled)")
plt.grid(True)
plt.tight_layout()
plt.show()

### For CNN (better for images)

''' python
from keras.layers import Conv2D, MaxPooling2D

model = Sequential()
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(28,28,1)))
model.add(MaxPooling2D((2, 2)))
model.add(Flattn())
model.add(Dense(64, activation='relu'))
model.add(Dense(10, activation='softmax'))
'''
```

```
# Step 1: Import necessary Libraries
import numpy as np
import matplotlib.pyplot as plt
from keras.datasets import fashion_mnist
from keras.models import Sequential
from keras.layers import Dense, Flatten
from keras.utils import to_categorical

# -----Step 2: Load the Fashion MNIST dataset-----
(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()
```

```
# -----Step 3: Preprocess the data-----
```

```
# Normalize pixel values to the range [0, 1]
x_train = x_train / 255.0
x_test = x_test / 255.0
```

```
# One-hot encode the labels (10 classes)
y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)
```

```
# -----Step 4: Create the model-----
```

```
model = Sequential()
```

```
# -----Step 5: Add Layers to the model-----
```

```
# Flatten 2D images (28x28) to 1D vectors
model.add(Flatten(input_shape=(28, 28)))
```

```
# Hidden Layer with 100 units and ReLU activation
model.add(Dense(100, activation='relu'))
```

```
# Output Layer with 10 units (softmax for multi-class classification)
model.add(Dense(10, activation='softmax'))
```

```
# -----Step 6: Compile the model-----
```

```
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
```

```
# -----Step 7: Train the model-----
```

```
# Set batch size to the full training set (batch gradient descent)
batch_size = len(x_train)
```

```
# Train the model for 100 epochs and validate on test data
history = model.fit(x_train, y_train_cat,
                    epochs=100,
                    batch_size=batch_size,
                    validation_data=(x_test, y_test_cat),
                    verbose=0)
```

```
# -----Step 8: Plot training and validation Loss-----
```

```
plt.figure(figsize=(8, 5))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Loss over Epochs')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
plt.show()
```

```
# -----Step 9: Display training history for each epoch-----
```

```
for i in range(100):
    print(f"Epoch {i+1}: "
          f"Loss = {history.history['loss'][i]:.4f}, "
          f"Val Loss = {history.history['val_loss'][i]:.4f}, "
          f"Accuracy = {history.history['accuracy'][i]*100:.2f}%, "
          f"Val Accuracy = {history.history['val_accuracy'][i]*100:.2f}%")
```

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder
```

```
# -----Step 1: Create the dataset-----
```

```
data = {
    'id': [1,2,3,4,5,6,7,8,9,10],
    'item': ['milk', 'sugar', 'chips', 'coffee', 'meat', 'chocos', 'juice', 'jam',
    'quantity': [2.0, 1.0, np.nan, 2.0, 4.0, 3.0, 1.0, np.nan, 3.0, 4.0],
    'price': [67.0, np.nan, 45.0, 45.0, 56.0, np.nan, 78.0, 65.0, np.nan, np.nan],
    'bought': [672.0, 453.0, 456.0, 672.0, 786.0, 345.0, 765.0, 665.0, np.nan, np.nan],
    'forenoon': [456.0, 234.0, 322.0, 564.0, 221.0, np.nan, np.nan, np.nan, 567.0,
    'afternoon': [np.nan, np.nan, np.nan, np.nan, np.nan, np.nan, 213.0, 344.0, 333.0, np.nan]
```

```
df = pd.DataFrame(data)
```

```
# -----Step 2: Handle missing values-----
```

```
# Replace missing values in 'price' with column mean
mean_price = df['price'].mean()
df['price'] = df['price'].replace(np.nan, mean_price)
```

```
# Replace missing 'quantity' with 0
df['quantity'] = df['quantity'].replace(np.nan, 0)
```

```
# Replace missing 'afternoon' values with mode
mode_afternoon = df['afternoon'].mode()[0]
df['afternoon'] = df['afternoon'].replace(np.nan, mode_afternoon)
```

```
# Replace missing 'forenoon' values with mode
mode_forenoon = df['forenoon'].mode()[0]
df['forenoon'] = df['forenoon'].replace(np.nan, mode_forenoon)
```

```
# -----Step 3: Convert categorical 'item' column to numeric-----
```

```
df['item'] = LabelEncoder().fit_transform(df['item'])
```

```
# -----Step 4: Replace missing 'bought' values using mean of same item group---
df['bought'] = df.groupby('item')['bought'].transform(lambda x: x.fillna(x.mean()))
```

```
# Required Imports: Lemmatization
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.corpus import stopwords, wordnet
from nltk.stem import WordNetLemmatizer
import string
import nltk
```

```
# Download required resources
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')
```

Step 3: Define Model Layers (This Changes the Most)

You build your model by **stacking layers** using 'Sequential()'.

For Image (Simple Dense Model)

python

```
model = Sequential()
model.add(Flatten(input_shape=(28, 28))) # Turns image into 1D
model.add(Dense(100, activation='relu')) # Hidden layer
model.add(Dense(10, activation='softmax')) # Output layer for 10 classes
```

Step 3: Lemmatization

Lemmatizer = WordNetLemmatizer()

for tokens in tokenized_sentences:

lemmatized = [lemmatizer.lemmatize(word) for word in tokens]

lemmatized_sentences.append(lemmatized)

print("\nLemmatized Tokens:")

for lemma in lemmatized_sentences:

print(lemma)

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import r2_score, mean_squared_error
```

```
# Load the dataset from CSV file
df = pd.read_csv('cat_breeds_clean.csv', delimiter=';')
```

```
# Display the first few rows of the dataset
df.head()
```

```
# -----STEP 1: Plot BoxPlots -----
```

```
# Define categorical features to analyze their effect on Weight
cat_features = ['Gender', 'Neutered_or_spayed', 'Fur_colour_dominant', 'Fur_pattern']
```

```
# Set the figure size for plotting
plt.figure(figsize=(15, 10))
```

```
# Loop through categorical features and plot Weight vs each feature
```

```
for i, feature in enumerate(cat_features, 1):
    plt.subplot(2, 2, i) # Create a subplot for each feature
    sns.boxplot(x=df[feature], y=df['Weight']) # Boxplot to show weight distribution by category
    plt.title(f'Weight vs {feature}')
    plt.xticks(rotation=45) # Rotate x-axis labels for readability
```

```
plt.tight_layout() # Adjust spacing between plots
plt.show()
```

```
# -----STEP 2: Prepare Features & Labels -----
```

```
# Define feature columns and target Label
features = ['Age_in_months', 'Body_length', 'Neutered_or_spayed', 'Owner_play_time_minutes']
label = 'Weight'
```

```
# Convert 'Neutered_or_spayed' to numeric (0/1)
df['Neutered_or_spayed'] = df['Neutered_or_spayed'].astype(int)
```

```
# Drop rows with any missing values in selected columns
data = df[features + [label]].dropna()
```

```
# Separate features (X) and Label (y)
X = data[features]
y = data[label]
```

```
# -----STEP 3: Split the Data -----
```

```
# 80:20 split
X_train_80, X_test_20, y_train_80, y_test_20 = train_test_split(X, y, test_size=0.2)
```

```
# 70:30 split
X_train_70, X_test_30, y_train_70, y_test_30 = train_test_split(X, y, test_size=0.3)
```

```
# -----STEP 4: Train the Models -----
```

```
# Create Linear Regression models
model_80 = LinearRegression()
model_70 = LinearRegression()
```

```
# Train the models
model_80.fit(X_train_80, y_train_80)
model_70.fit(X_train_70, y_train_70)
```

```
# -----STEP 5: Make Predictions -----
```

```
# Predict weights using test sets
y_predict_20 = model_80.predict(X_test_20)
y_predict_30 = model_70.predict(X_test_30)
```

```
# -----STEP 6: Evaluate the Models -----
```

```
# Create dictionary to store evaluation results for both splits
result = {}

"70-30": {
    "R2_Score": r2_score(y_test_30, y_predict_30), # R-squared for 70-30 split
    "MSE": mean_squared_error(y_test_30, y_predict_30) # Incorrect: MSE should match 70-30 split
},
"80-20": {
    "R2_Score": r2_score(y_test_20, y_predict_20), # R-squared for 80-20 split
    "MSE": mean_squared_error(y_test_20, y_predict_20) # MSE for 80-20 split
}
```

```
# Required Imports
```

```
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.corpus import stopwords
from nltk.stem.porter import PorterStemmer
import string
import nltk
```

```
# Download necessary resources from NLTK
nltk.download('punkt')
nltk.download('stopwords')
```

```
# Input Blog Text
```

```
blog = """Artificial Intelligence (AI) is rapidly transforming the job market. While it's true that some repetitive and routine jobs may be automated, AI is expected to augment our abilities. Rather than replacing humans entirely, AI is expected to augment our abilities. The future of work will belong to those who can adapt, learn new skills, and
```

```
# -----Step 1: Split blog into individual sentences-----
```

```
sentences = sent_tokenize(blog)
print("List of Sentences:")
for sentence in sentences:
    print(sentence)
```

```
# -----Step 2: Tokenize each sentence into words-----
```

```
tokenized_sentences = [word_tokenize(sentence) for sentence in sentences]
print("\nTokenized Sentences:")
for tokens in tokenized_sentences:
    print(tokens)
```

```
# -----Step 3: Apply stemming to each token-----
```

```
ps = PorterStemmer()
stemmed_sentences = []
```

```
for tokens in tokenized_sentences:
    stemmed = [ps.stem(word) for word in tokens]
    stemmed_sentences.append(stemmed)
```

```
print("\nStemmed Tokens:")
for stemmed in stemmed_sentences:
    print(stemmed)
```